

Skip List
–Destroy List
–Time Complexity Analysis

Program:

```
void destroyList(struct SkipList *lst) {
    // Start at the header
    struct Node *current = lst->header;
    struct Node *next;

    // We only need to traverse level 0 (the "local train" level)
    // because it is the only level guaranteed to contain EVERY node.
    while (current != NULL) {
        // 1. Save the pointer to the next node
        next = current->forward[0];

        // 2. Free the dynamically allocated array of pointers
        free(current->forward);

        // 3. Free the node itself
        free(current);

        // 4. Move to the next node
        current = next;
    }

    // Finally, free the main Skip List structure
    free(lst);
    cout << "Memory successfully freed. Skip List destroyed." << endl;
}

...

case 5:
    cout << "Destroying the current list..." << endl;
    destroyList(list);
    // CRITICAL: We must give the program a fresh, empty list
    // so it doesn't crash if the user tries to insert again.
    break;
```

Time Complexity Analysis

Unlike the **Insertion**, **Search**, and **Deletion** modules, the **destroyList** function

does not rely on probability, coin flips, or navigating express lanes.

Because its only goal is to completely wipe the data structure from memory, it uses a straightforward, brute – force sweep.

Because of this, the Time Complexity for the Destroy module does not fluctuate. **The Best, Worst, and Average** cases are all exactly **$O(n)$** .

The Traversal Strategy: Level 0 Only

To destroy the list, the algorithm must ensure it does not miss a single node.

- **The Code:** It initializes a pointer at the header and uses a **while (current != NULL)** loop to walk forward. To move to the next node, it strictly uses **next = current → forward[0]**.
- **The Logic:** Level 0 (the ground floor) acts as the "local train". It is the only level mathematically guaranteed to contain every single node in the Skip List. By ignoring the upper – level express lanes entirely and just walking across Level 0, the algorithm guarantees it will visit exactly n nodes (plus the dummy header) one by one.
- **The Complexity:** Because it visits every node exactly once, the traversal takes **$O(n)$** time.

The Deallocation Strategy: The $O(1)$ Two – Step Wipe

At every single node, the program performs a two – step freeing process:

- 1. **$free(current \rightarrow forward)$** : The **$forward$** array holding all the pointers for that node was dynamically allocated as a single, contiguous array in memory. The program does not need a loop to delete **$forward[0]$** , then **$forward[1]$** , etc. It hands the starting address of the chunk to the operating system, which deallocates the entire array in one single **$O(1)$** operation.*
- 2. **$free(current)$** : Once the pointer array is gone, this command wipes out the node structure itself (which holds the **key**), also in **$O(1)$** time.*

Final Calculation

*The algorithm visits n nodes. At each node, it performs the two – step **$O(1)$** deallocation. Finally, it drops out of the loop and executes **$free(lst)$** to destroy the main Skip List container.*

$$n \text{ nodes} \times O(1) \text{ deletion operations} = O(n)$$

*Therefore, no matter how perfectly balanced or catastrophically unbalanced the Skip List is, the time complexity to destroy it is universally and strictly **$O(n)$** .*
